02-750 Week 8

Automation of Scientific Research

Aidan Jan

March 17, 2026

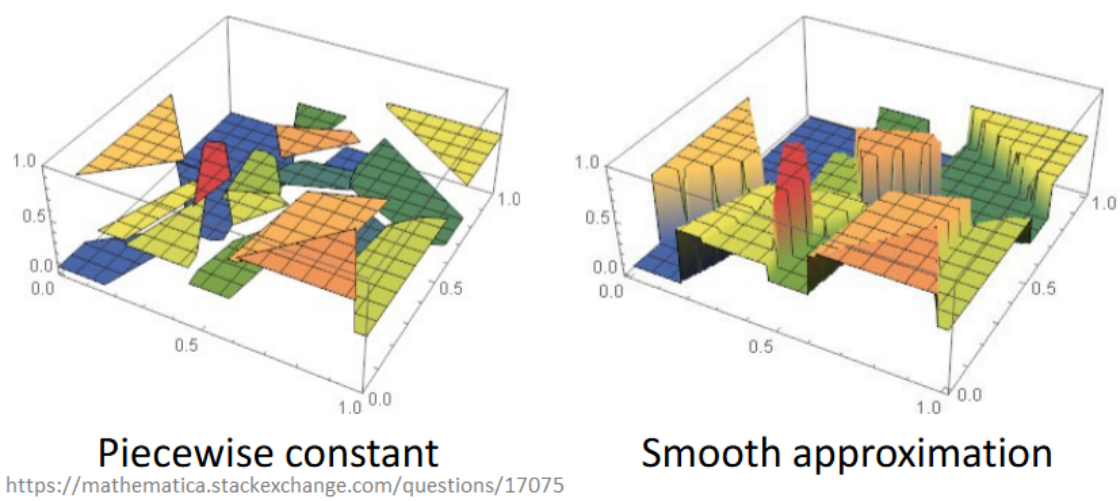
PLAL Algorithm

Key Features:

- PLAL is an algorithm for learning **binary and multiclass classifiers**
- It is a Type II algorithm (i.e., cluster exploitation)
- It is intended to be used with pool-based data access model
- It handles non-separable data
- It is mellow
- Guarantees:
  - PLAL will converge to near-optimal labelings
  - PLAL bounds the number of samples needed to reach near-optimal labeling

Model

- Classification models encode **piecewise constant functions**
- The theorems associated with PLAL are based on assumptions about the properties of a smooth approximation of the piecewise constant function
  - Note: PLAL does not actually construct the smooth approximation



Lipschitz Continuity

PLAL assumes that the data are "Probabilistic Lipschitz"

- Informally, this means that the labels change smoothly nearly everywhere throughout the input domain, $\mathcal{X}$, which guarantees that the instances within each cluster will **probably** have the same label

A Lipschitz continuous function is one where rate of change of the function is bounded by a constant, everywhere.

$$|f(x) - f(x')| \leq 1/\lambda |x - x'|, \forall x, x' \in [l, u]; \lambda > 0$$

- Here, $\frac{1}{\lambda}$ is the **Lipschitz constant**

The PLAL algorithm makes assumptions about the continuity of the labeling function, $f : \mathcal{X} \to \mathcal{Y}$. Importantly, Lipschitz continuity lets us compute **upper** and **lower bounds** on the value of the function in *any* closed interval $[a, b]$.

- If the labeling function is Lipschitz-continuous, the data is **clusterable**
- Distance between clusters is approximately $\frac{1}{\lambda}$
- Label boundaries go through low-density regions

Probabilistic Lipschitz Continuity

Probabilistic Lipschitz relaxes the notion of 'pure' Lipschitz continuity; by allowing some violations, it bounds the probability such violations occur

- i.e., the clusters are **nearly** label-homogeneous
- $P(\text{violates Lipschitz continuoity}) \leq \phi(\lambda), \forall \lambda > 0$
- Intuition: if we center a ball of radius λ on a cluster with label l, then the probability of finding instances in that ball that have a different label is bounded by some function $\phi(\lambda) : \mathbb{R} \to [0, 1]$

PLAL labeling procedure

- First, use space partitioning trees to create the clusters (hierarchical clustering). These clusters do not necessarily need to be correct, and they cluster based on features, not labels.
- In each cluster, check the labels of already-labelled points. If the found labels are all the same, classify all unlabeled points in that cluster with the label.
- If the labels are not all the same, then split the cluster into smaller clusters, and repeat.

PLAL Guarantees

Consistency:

- Given a set S containing m instances sampled iid and parameters ϵ and δ, PLAL labels at least $(1-\epsilon)m$ instances correctly, with probability $(1 - \delta)$

Sample Complexity:

- Recall that probabilistic Lipschitzness is defined as:

$$P(\text{violates Lipschitz continuity}) \leq \phi(\lambda)$$

- The sample complexity of PLAL depends on the nature of ϕ and the spatial tree
- For **dyadic trees**, where cells are partitioned by halving the dimensions,
  - If $\phi(\lambda) = \lambda^n$, the sample complexity is $O\left(m^{\frac{d}{n+d}} \left(\frac{1}{\epsilon}\right)^{\frac{n}{n+d}}\right)$, where d is the dimensionality of the data
  - If $\phi(\lambda) = \exp\left(-\frac{1}{\lambda}\right)$, the sample complexity is $O\left(\frac{1}{\epsilon}\right)$

Summary of Type II Algorithms

- Type II query selection techniques typically perform **cluster analysis** or related methods as a preprocessing step to reveal patterns or structure in the data pool, under the assumption that the instances assigned to a given cluster will probably have the same label
- The DH and PLAL algorithms build a hierarchy over the pool and use it to guide sample selection, but do so in different ways
- Assuming that the initial pool is iid, DH and PLAL produce an unbiased set of labels, because they label everything (either by going to the oracle, or by inferring labels through cluster assignment)
- Both methods are consistent and have favorable label complexities, relative to standard supervised learning

Proactive Learning

Each query selection technique we have covered made 4 assumptions:

- There is one oracle
- Oracles always give the correct label
- Oracles always give an answer
- Oracles have fixed costs (i.e., each call has the same price)

Proactive Learning relaxes these assumptions.

- The different oracles may represent different kinds of experiments, or different people
- Experiments may occasionally give the wrong answer
- Experiments may simply fail and produce no result whatsoever
- Different types of experiments have different costs

Key features:

- It presents three algorithms for learning classifiers 'proactively'
- It handles non-separable data
- It is intended for **pool-based selective sampling**
- It combines aspects of heuristic and Type II algorithms
- It is **aggressive**
- Guarantees: None

Notation:

- Let B be our labeling budget
- Let UL be a pool of unlabeled instances
- Let $V(x)$ be the estimated value of labeling instance $x \in UL$. $V(\cdot)$ can be any query selection criterion
- Let K be the set of oracles
- Let C_k be the cost of the kth oracle
- Let $U(x, k)$ be the utility of labeling instance x using oracle k. The three algorithms define U in different ways

Proactive Learning: Overall Strategy

Pseudocode for the abstract proactive learning strategy

```
while budget B is not exhausted:
    Let (x*, k*) = argmax([U(x, k) for x in UL, k in K])
    Pay oracle k* to label instance x*
```

Notice that this is an aggressive strategy. The choice of x and k depend on one another, so we need to rank (x, k) pairs.

Three Scenarios

The authors consider three scenarios:

1. One reliable oracle and one **reluctant** oracle.
    - Reluctant means that it sometimes won't answer, because it is not confident about the true label
2. One reliable oracle and one **fallible** oracle
    - The fallible oracle sometimes returns incorrect labels
3. One fixed cost oracle and one **variable cost** oracle

Each scenario gives rise to a different concrete algorithm with a different way to define the utility function, U.

Setup

- Use real datasets
    - Train reliable oracle with all available data
        - Always answers and is always correct
    - Train unreliable oracle with only a small portion
        - If its estimated uncertainty is in a given range, fallible returns a random answer, reluctant returns no answer.
- Pick an active learning function $V(x)$ (e.g., uncertainty sampling)

Proactive Learning: Scenario 1

One reliable oracle and one reluctant oracle

- The reliable oracle always answers and always gives the correct label
- The reluctant oracle sometimes doesn't answer, but when it does, it gives the correct label
- The reliable oracle is more expensive

The utility for scenario 1 is defined as:

$$U(x, k) = P(ans|x, k)\frac{V(x)}{C_k}$$

where $P(ans|x, k)$ is the probability that the oracle responds.

- Note: $P(ans|x, k) = 1$, if k is the reliable oracle

The key challenge for scenario 1 is estimating $P(ans|x, k)$

- The authors estimate this using a centroid-based clustering algorithm (e.g., K-means)
  - A portion of the budget $B_c < B$ is set aside for estimating $P(ans|x,k)$
  - $p = round(B_c/C_{reluctant})$
  - The *reluctant* oracle is asked to label one instance, x_{ct}, from each cluster
    - x_{ct} is the instance closest to the cluster's centroid
  - Note: the reluctant oracle may return a label for x_{ct}, or it may not
- The algorithm assumes that the probability of getting an answer for instance x is proportional to its distance to the nearest x_{ct} and the reluctant oracle's responsiveness on that x_{ct}

$$\hat{P}(ans|x, reluctant) = \frac{0.5}{Z} \exp\left(\frac{\mathbb{I}(x_{c_t}, y_{c_t})}{2} \log\left(\frac{\max_d - \|x_{c_t} - x\|}{\|x_{c_t} - x\|}\right)\right), \quad \forall x \in S(x_{c_t})$$

  where $S(x_{c_t})$ is the set of instances closest to centroid x_{c_t}, $\max_d$ is the maximum distance between any centroid and *any* other instance, and $\mathbb{I} \in \{-1, 1\}$ is an indicator function such that $\mathbb{I}(x_{c_t}, y_{c_t}) = 1$, if the reluctant oracle returned a label for x_{c_t}, and -1, if it did not

Proactive Learning: Scenario 2

One reliable oracle and one fallible oracle

- The reliable oracle always answers and always gives the correct label
- The fallible oracle sometimes doesn't return the correct label
- The reliable oracle is more expensive

The utility for scenario 2 is defined as:

$$U(x,k) = P(correct|x,k)\frac{V(x)}{C_k}$$

The key challenge for scenario 2 is estimating $P(correct|x,k)$

- The strategy for estimating $P(correct|x,k)$ is very similar to the one used to estimate $P(ans|x,k)$ in scenario 1.
  - A portion of the budget, $B_c < B$ is set aside for estimating $P(correct|x,k)$ using k-means clustering, or similar
  - $p = round(\frac{B_c}{C_{fallible}})$
  - The fallible oracle is asked to label one instance, x_{ct}, from each cluster.
  - Note: The fallible oracle must return the conditional probability of the label. i.e., faliible oracle returns $P(y|x_{c_t})$

Proactive Learning: Scenario 3

One uniform cost oracle and one variable cost oracle

- Both oracles always return the correct labels
- The uniform cost oracle always charges the same price
- The variable cost oracle charges proportional to the difficulty of the query

The utility for scenario 3 is defined as:

$$U(x,k) = V(x) - C_k(x)$$

- Note: assume that V and C are normalized to the same range

The price charged by the variable cost oracle is as follows:

$$C_{variable}(x) = 1 - \frac{\max_{y \in \dagger} P(y|x) - 1/|\dagger|}{1 - 1/|\dagger}$$

where $\dagger$ is the set of labels, and $P(y|x)$ is the conditional probability of label y, as determined by the variable cost oracle. Thus, the cost is inversely proportional to the conditional probability of the most likely label

- Note: $P(y|x)$ is only known by the variable cost oracle. We assume that the oracle pre-computes the prices for each $x \in UL$, then gives the 'price list' to the user.

Summary: Proactive Learning

- Proactive Learning relaxes the assumption that there is a single oracle and allows for the possibility that one or more of the oracles is reluctant, unreliable, or has variable costs
- The benefits of Proactive Learning (relative to simply always selecting the 'best' oracle) depends on (i) the cost ratios among oracles, and (ii) the degree to which the imperfect oracles deviate for the perfect one

Active Feature Value Selection

Real-world data sets often have **missing feature values**. Each feature may require running a separate experiment

Strategies for handling missing data:

- Ignore it: just use a base learner that natively handles missing data
  - ex. decision trees
  - But, some learning algorithms cannot (easily) ignore missing data
- Use it: for categorical variables, treat 'missing' as a category
  - But, this won't work for numeric data
- Eliminate it: remove incomplete features (columns) or instances (rows)
  - But, this means throwing data away
  - What do you do if every feature (or instance) has a missing value?
- Imputation: predict the missing data features
  - Simple imputation methods (ex. use mean/median/mode of observed values)
  - Complex imputation methods (ex. density estimation; generative models)
  - But, this means 'inventing' data
- Integration: "sum out" the missing values
  - But, this can be very expensive, and requires a distribution over each feature
- Compensation: pay to acquire the missing feature values

We consider two cases

- Case 1: The labels for all instances are known
- Case 2: Some labels are not known (and unknown labels are treated as missing features)

Data access model: Pool-based sampling. Query selection: expected risk minimization.

Algorithm 1 (known labels)

- Let F be an (incomplete) m by n instance-feature matrix. n is the number of measurable features, m is the number of instances in the pool
- $F_{i,j}$ is the j-th feature for the i-th instance. Any $F_{i,j}$ can be missing.
- $Y = \{y_i : i = 1, \ldots, m\}$ is the set of class labels
- $T = (F, Y)$ is the pool of training data
- L is the learning algorithm. Importantly, L must be capable of handling missing values
- $\beta \geq 1$ is the batch size
- C is the cost matrix, where $C_{i,j}$ is the cost to acquire the j-th feature for the i-th instance. Costs need not be uniform.

```
Initialize set of possible queries Q = \
    {q_ij for i in range(m) for j in range(n), if F_ij is missing}
While stopping criterion not met:
    Induce a classifier, M = L(T)
    for q_ij in Q:
        compute score(q_ij, c_ij, L, T)
    Select the subset S of size beta
    for q_ij in S:
        Acquire values for F_ij
        Remove q_ij from Q
Return M = L(T)
```

Scoring Function

The authors propose the following function to estimate the **expected utility** of a particular query $q_{i,j}$

$$score(q_{i,j}, c_{i,j}, L, T) = E(q_{i,j}) = \sum_{k=1}^{K} \mathcal{U}(F_{i,j} = v_k) P(F_{i,j} = v_k)$$

- $\mathcal{K}$ is the number of distinct values the j-th feature can take on
- $\mathcal{U}$ is a measure of the feature's utility for this instance
- $P(F_{i,j} = v_k)$ is the probability the j-th feature is v_k for instance i
  - For the first algorithm, $P(F_{i,j} = v_k) = P(F_{i,j} = v_k | y)$ is the class-conditional probability of the feature (estimated using the known values)

Utility

A query's utility is the marginal improvement in performance per unit cost

$$\mathcal{U}(F_{i,j} = v_k) = \frac{\mathcal{A}(F, F_{i,j} = v_k)}{C_{i,j}}$$

- $\mathcal{A}(F, F_{i,j} = v_k)$ is a function that measures the marginal improvement of the model if we measure feature $F_{i,j}$ and discover that it has value v_k

To measure marginal improvement, $\mathcal{A}$,

- The algorithm assumes that we have a set of hold-out instances $T_H = (F_H, Y_H)$ in addition to the training data, $T = (F, Y)$

- The data in T_H is not used to train models or to estimate $P(F_{i,j} = v_k)$, so we can use it to determine whether a model trained under the assumption that $F_{i,j} = v_k$ is better or worse than one trained with $F_{i,j} = v'_k$, or one trained leaving $F_{i,j} =?$
- Let $LG(x) = \sum_y -\delta(y) \log \hat{f}_{F'}(y|x)$ be the log-gain (aka cross-entropy)
  - Here, $\hat{f}_{F'}(y|x)$ is the estimated posterior class probability a model trained on set F' where we've temporarily assigned missing value $F_{i,j}$ to some v_k
  - δ is an indicator function that is 1, if y is the correct label, and zero otherwise
  - Small values of LG indicate that the model places high probability on the correct label for some instance x
- Let $\mathcal{A}(F, F_{i,j} = v_k) = \sum_{e \in T_H} LG(x_e)$ be the total log-gain if $F_{i,j} = v_k$

Why do we use log-gain instead of classification error?

- Classification error isn't sensitive to the confidence of the predictions.

Managing Computational Complexity

- A brute-force implementation of this algorithm would require training $O(mnK)$ models, one for each possible value of each missing feature
- Training and evaluating a model requires $\Omega(mn)$ time
- These costs could become prohibitive for large values of m, n, or K

The authors address the complexity problem by sampling from set Q

- Let $1 \leq \alpha \leq mn/\beta$ be the **exploration parameter**
- The algorithm will only compute utilities for $\alpha\beta$ elements from Q each round, and it will select β elements from this sampled subset to query
- They call this the **Sampled Expected Utility** (SEU)

Sevaral variants of SEU are possible

- SEU-US: samples are chosen uniformly at random
- SEU-ES: samples are chosen using **error sampling**. That is, the probability of being sampled is 'high' if the current model mis-labels a given instance (recall, labels are known), otherwise it is 'low'

Variants of SEU tested:

- SEU-US: random sampling
- SEU-ES: error sampling
- SEU-Accuracy: use A = classification accuracy (rather than log-gain)
- SEU-Frequency: P is not class-conditional
- SEU-DT: P from tree induction (decision tree)

Out of those tested, SEU-ES works the best, overall.

Algorithm 2 (Missing labels)

The main difference here is that now any $F_{i,j}$ or y_i can be missing.

- The second algorithm treats labels as just another kind of feature
- The main differences between the algorithms is how they estimate $P(F_{i,j} = v_k)$ and how they rank missing values
  - The 2nd algorithm uses classification trees to estimate $P(F_{i,j} = v_k)$, rather than using class-conditional probabilities
  - Roughly, $P(F_{i,j} = v_k) = \sum_y P(F_{i,j} = v_k|y)P(Y_i = y)$
  - Classification trees are also used to estimate $P(Y_i = y)$, for missing Y_i
  - Missing values are ranked using a somewhat different function that considers the uncertainty in the class-label

Summary: Active Feature Value Acquisition

- Real-world data sets often contain missing feature values (ex. when different features are measured using different experiments)
- Active Feature Value Acquisition generalizes active learning to handling both missing feature values and missing labels
  - The paper by Saar-Tsechansky, *et al* uses a heuristic query selection strategy, but the results are promising on benchmark data sets
  - Developing Active Feature Value Acquisition algorithms with formal guarantees remains an open problem

Reinforcement Learning

Reinforcement learning is modeled as a **Markov Decision Process**

- A Markov decision process is a 4-tuple (S, A, P_a, R_a), where
  - S is a set of states called the *state space*
  - A is a set of actions called the *action space*. A_s is the set of actions available from state s
  - $P_a(s, s') = P(s_{t+1} = s'|s_t = s, a_t = a)$ is the probability that action a in state s at time t will lead to state s' at time $t + 1$
  - $R_a(s, s')$ is the immediate reward (or expected immediate reward) received after transitioning from state s to state s', due to action a
- A policy function π is a potentially probabilistic mapping from state space S to action space A.
- **Objective:** choose a policy π that will maximize some cumulative reward

Active Learning as Decision Making

There are many similarities between active learning and reinforcement learning

Active Learning	**Reinforcement Learning**
Heuristic selection	Learned selection 'action'
Accuracy	Reward feedback
Budget	Budget
Simulation for evaluation	Simulation for learning (and evaluation)

Why not frame active learning as reinforcement learning?

Learning how to Active Learn

Goal of Active Learning: Select a small subset of data for annotation such that a classifier learned on the data is highly accurate

- Usually done using heuristic selection methods but performance of heuristics varies between datasets

Key Idea

- Introduce a novel formulation by reframing the active learning as a reinforcement learning problem and explicitly learning a data selection policy, where the policy takes the role of the active learning heuristic